fn then_saturating_cast

Michael Shoemate

This proof resides in **"contrib"** because it has not completed the vetting process.

Proves soundness of the implementation of `then_saturating_cast` in `mod.rs` at commit f5bb719 (outdated[1]).

1 Hoare Triple

Precondition

Compiler-Verified

- Generic `TO` implements trait `SaturatingCast<IBig>`

User-Verified

None

Pseudocode

```
def then_saturating_cast(modular: bool) -> Function[Vec[IBig], Vec[TO]]:
    return Function.new(
        lambda x: [
            TO.saturating_cast(x_i + IBig.from_(TO.MIN_FINITE)) if modular else TO.saturating_cast(x_i)
            for x_i in x
        ]
    )
```

Postcondition

Theorem 1.1. For every setting of the input parameters (`TO`) to `then_saturating_cast` such that the given preconditions hold, `then_saturating_cast` raises an error (at compile time or run time) or returns a valid postprocessor. A valid postprocessor has the following property:

1. (Data-independent errors). For every pair of members x and x' in `input_domain`, `function`(x), `function`(x') either both raise the same error, or neither raise an error.

Proof. Since `TO.saturating_cast` is infallible, and addition by the constant `TO::MIN_FINITE` is infallible on `IBig`, the function is infallible. Therefore the function cannot raise data-dependent errors.

In the non-modular branch, the function is coordinatewise saturating cast. In the modular branch, it first translates each coordinate by the constant `TO::MIN_FINITE` and then applies the same infallible saturating cast. Both branches are deterministic postprocessings, so the function is a valid postprocessor. □

[1] See new changes with `git diff f5bb719..40879b13 rust/src/measurements/noise/nature/integer/mod.rs`